Product Requirements Document

Unified Marine Intelligence & Ecosystem Reasoning Platform

Target Problem Statement: SIH PS 20 — AI-Driven Unified Data Platform for Oceanographic, Fisheries and Molecular Biodiversity Insights

Document Purpose: This document is the authoritative implementation specification for building the first complete prototype of the platform.

Primary implementation target: A working web application that ingests heterogeneous marine datasets, normalizes them into a unified spatial-temporal-taxonomic data model, performs scientific analytics, uses AI agents for cross-domain reasoning, and presents evidence-backed marine ecosystem insights through an interactive interface.

---

1. Product Vision

Build a marine intelligence platform that allows a researcher, policymaker, environmental analyst, fisheries expert, or student to ask natural-language questions about marine ecosystems and receive:

1. A scientifically computed answer.
2. Relevant observations from multiple marine domains.
3. Cross-domain relationships between oceanography, fisheries, biodiversity, and molecular biodiversity.
4. Interactive maps and charts.
5. Source provenance.
6. Claim-level evidence.
7. Explicit uncertainty.
8. A clear distinction between correlation, association, and causation.

The platform must **not** simply be a chatbot over documents.

The core value proposition is:

> **Unify heterogeneous marine observations into a common analytical layer and use evidence-grounded AI reasoning to produce explainable ecosystem insights.**

---

2. Product Goals

2.1 Primary goals

The completed MVP must be capable of:

- Ingesting public oceanographic data.

- Ingesting public fisheries data.
- Ingesting marine biodiversity observations.
- Ingesting or representing molecular biodiversity data.
- Normalizing species/taxonomy.
- Normalizing time.
- Normalizing coordinates.
- Storing data in a unified database.
- Performing statistical and geospatial analysis.
- Answering natural-language marine questions.
- Routing questions to appropriate datasets.
- Using multiple domain-specific AI agents.
- Combining agent evidence.
- Performing claim-level verification.
- Showing provenance for generated claims.
- Visualizing observations on maps.
- Visualizing temporal trends.
- Showing ecosystem relationships.
- Providing an uncertainty/confidence indicator.
- Supporting a constrained regional dataset for development and demo purposes.
- Working even when external datasets/APIs are unavailable by using local demo data.

---

3. Non-Goals

The initial implementation must NOT attempt to:

- Train a new large language model.
- Train a foundation model for oceanography.
- Process the entire global Argo archive.
- Download the complete global OBIS archive.
- Process raw terabytes of sequencing data.
- Build a global real-time ocean forecasting system.
- Claim causal relationships from simple correlations.
- Replace marine scientists.
- Automatically make regulatory or policy decisions.
- Build a production-grade global distributed infrastructure.
- Require specialized GPU hardware.
- Require all external APIs to be available for the demo.

The architecture should be extensible toward these capabilities, but the MVP must remain feasible on ordinary developer hardware.

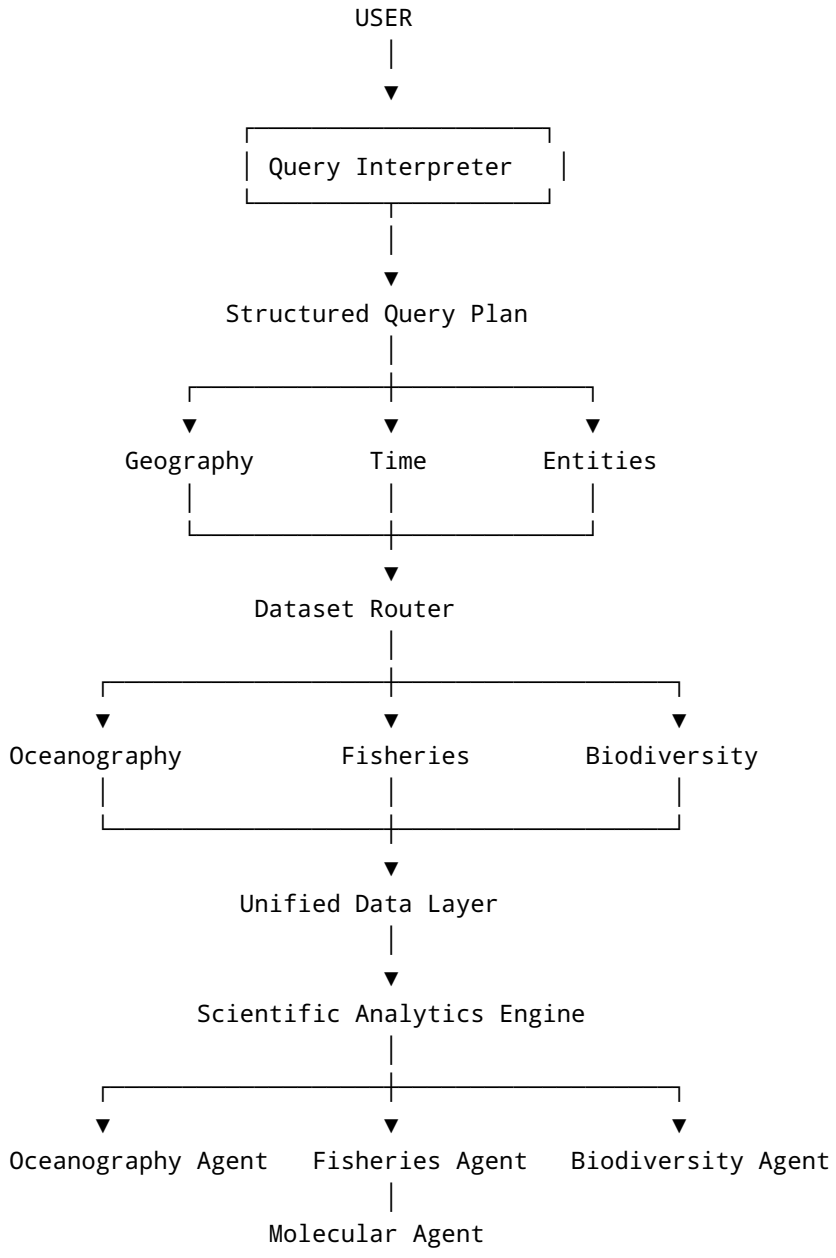
---

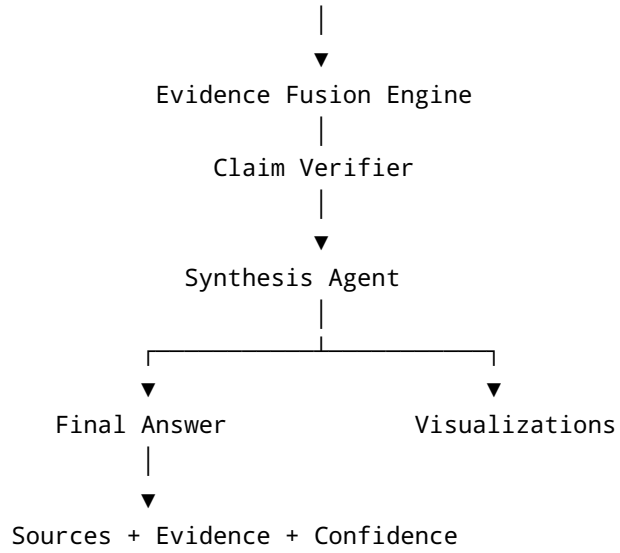
4. Core Product Concept

The platform is composed of six major layers:

```
User
  ↓
Query Understanding
  ↓
Unified Data Layer
  ↓
Scientific Analytics
  ↓
Collaborative AI Reasoning
  ↓
Evidence + Visualization + Final Insight
```

Detailed architecture:





---

5. Supported Domains

The initial system must support these four conceptual domains.

5.1 Oceanographic

Variables may include:

- Temperature
- Salinity
- Depth
- Pressure
- Dissolved oxygen
- Chlorophyll
- Coordinates
- Observation timestamp

Primary initial source:

Argo

Future sources:

- World Ocean Database
- Copernicus Marine Service

5.2 Fisheries

Variables may include:

- Species
- Catch
- Production
- Region
- Year
- Fishing area
- Abundance where available
- Survey observations where available

Primary initial source:

FAO FishStat

Future source:

ICES DATRAS

5.3 Marine Biodiversity

Variables may include:

- Species
- Taxon
- Latitude
- Longitude
- Observation date
- Depth
- Occurrence
- Abundance where available

Primary initial source:

OBIS

Future sources:

- EMODnet Biology
- GBIF where appropriate

5.4 Molecular Biodiversity

Variables may include:

- Sample ID
- Location
- Time
- Taxon
- Relative abundance
- Metagenomic profile
- Functional profile

Primary practical source:

MGnify

Future/advanced source:

- Tara Oceans
- NCBI SRA

Raw sequencing data must NOT be downloaded for the initial MVP unless specifically required.

---

6. Taxonomic Normalization

Use **WoRMS** as the common marine taxonomy reference layer.

Every species entity must support:

```
species_id
scientific_name
canonical_name
common_name
aphia_id
kingdom
phylum
class
order
family
genus
species
source_name
```

Example:

```
{
  "aphia_id": 127405,
```

```
  "scientific_name": "Thunnus albacares",
  "canonical_name": "Thunnus albacares"
}
```

The system must be able to map:

```
Yellowfin tuna
T. albacares
Thunnus albacares
```

to the same canonical species entity whenever the external taxonomy source confirms that mapping.

Do not use raw strings as the primary identity.

---

7. Spatial Normalization

All spatial observations must have a canonical representation:

```
latitude
longitude
depth_m
region_id
grid_cell_id
```

Create a configurable spatial grid.

For the MVP, use a configurable grid size such as:

```
0.25° × 0.25°
```

Do not hard-code the grid size.

Every observation should be assignable to a `grid_cell_id`.

The system must support:

- point observations
- bounding-box queries
- radius queries
- region queries
- grid-cell aggregation

The primary initial demo region should be configurable through environment/configuration.

Recommended default:

Arabian Sea / Western Indian Ocean region

Do not make the application globally dependent on that region.

---

8. Temporal Normalization

All timestamps must be internally stored in UTC.

Required derived fields:

```
year
month
quarter
season
```

The database must support:

- exact timestamp queries
- year ranges
- monthly aggregation
- seasonal aggregation
- rolling-window calculations

---

9. Unified Observation Model

All source datasets should be transformed into a compatible internal model.

Create a generalized observation representation:

```
{
  "observation_id": "uuid",
  "dataset_id": "argo",
  "domain": "oceanography",
  "timestamp": "2025-04-10T12:00:00Z",
  "latitude": 18.92,
  "longitude": 72.82,
  "depth_m": 40,
  "species_id": null,
  "variable": "temperature",
  "value": 27.4,
  "unit": "degC",
  "quality_flag": "valid",
  "source_record_id": "external-id",
  "source_url": "...",
```

```
  "metadata": {}
}
```

This generalized representation may coexist with domain-specific tables.

---

10. Database

10.1 Technology

Use:

PostgreSQL + PostGIS

Do not introduce multiple databases unless necessary.

The initial MVP should use one PostgreSQL instance.

---

11. Database Schema

Create at minimum:

`datasets`

```
id
name
domain
description
source_url
license
version
retrieved_at
metadata
```

`species`

```
id
aphia_id
scientific_name
canonical_name
common_name
taxonomy_json
```

locations

```
id
latitude
longitude
grid_cell_id
region_id
geometry
```

ocean_measurements

```
id
dataset_id
location_id
timestamp
depth_m
variable
value
unit
quality_flag
source_record_id
metadata
```

biodiversity_observations

```
id
dataset_id
species_id
location_id
timestamp
depth_m
observation_type
abundance
count
source_record_id
metadata
```

fisheries_records

```
id
dataset_id
species_id
region_id
year
catch
```

```
production
unit
source_record_id
metadata
```

molecular_samples

```
id
dataset_id
sample_identifier
location_id
timestamp
species_id
abundance
measurement_type
value
unit
source_record_id
metadata
```

evidence_records

```
id
query_id
source_type
source_id
claim_id
evidence_text
numeric_value
method
confidence
metadata
```

queries

```
id
user_query
structured_query
created_at
```

claims

```
id
query_id
claim_text
```

```
claim_type
status
confidence
```

`claim_evidence`

```
id
claim_id
evidence_id
relationship
weight
```

---

12. Data Ingestion Architecture

Every external dataset must have a connector.

Recommended structure:

```
backend/
  ingestion/
    argo/
      downloader.py
      parser.py
      transformer.py
    obis/
      downloader.py
      parser.py
      transformer.py
    fishstat/
      downloader.py
      parser.py
      transformer.py
    mgnify/
      downloader.py
      parser.py
      transformer.py
    worms/
      client.py
```

Each connector must implement a common interface:

```
class DatasetConnector:
    def download(self, config): ...
    def validate(self, raw_data): ...
```

```
    def transform(self, raw_data): ...
    def load(self, records): ...
```

---

13. Data Ingestion Requirements

Each connector must support:

- retry
- timeout
- logging
- validation
- missing-data handling
- duplicate detection
- provenance recording
- version metadata
- geographic filtering
- temporal filtering

No ingestion script should assume the entire dataset can fit into RAM.

Use streaming/chunking where appropriate.

---

14. Dataset Availability Fallback

External APIs are unreliable.

The application must have:

Mode A — External data

Uses current/public source data.

Mode B — Local snapshot

Uses previously downloaded subset files.

Mode C — Demo data

Uses a small synthetic dataset generated by the project.

The UI should display:

```
Data Mode: Demo Snapshot
```

or

```
Data Mode: Public Dataset Snapshot
```

Do not silently pretend synthetic/demo data is live scientific data.

---

15. Synthetic Demo Dataset

Create a reproducible generator.

The generator should create:

- ocean temperature measurements
- salinity
- species observations
- fisheries records
- molecular samples
- realistic time series
- realistic coordinates
- cross-domain relationships

It must intentionally support demonstration scenarios such as:

```
temperature increase
species decline
increased fishing pressure
biodiversity shift
```

Use a fixed random seed.

The generator must be deterministic.

---

16. Scientific Analytics Engine

Do not use an LLM to perform numerical calculations.

Implement a deterministic analytics service.

Required operations:

Trend

Calculate:

- mean
- median

- min
- max
- slope
- percentage change

Time-series

Support:

- monthly aggregation
- annual aggregation
- rolling mean
- seasonal aggregation

Correlation

Support:

- Pearson correlation
- Spearman correlation where appropriate

The API must clearly label correlation as correlation.

Never generate:

> "X caused Y"

from correlation alone.

---

17. Anomaly Detection

Implement lightweight statistical anomaly detection.

At minimum support:

```
z-score
rolling baseline
IQR-based detection
```

Example:

```
Current temperature = 29.5
Historical mean = 27.1
Std = 0.8
```

```
z = (29.5 - 27.1) / 0.8
```

Return:

```
{
  "anomaly_score": 3.0,
  "classification": "high",
  "baseline_period": "2018-2024"
}
```

ML anomaly detection such as Isolation Forest may be added later but is not required for MVP.

---

18. Spatial Analytics

Implement:

- species occurrence density
- heatmaps
- environmental grids
- regional averages
- spatial joins
- nearby-observation retrieval

Examples:

```
GET /api/analytics/species/{id}/distribution
GET /api/analytics/ocean/heatmap
GET /api/analytics/region/{region_id}/summary
```

---

19. Query Understanding

The natural-language input:

> "How has yellowfin tuna abundance changed in the Arabian Sea since 2021 and could temperature be a factor?"

must be converted to structured JSON.

Example:

```
{
  "intent": "trend_and_relationship",
```

```
  "entities": {
    "species": ["Thunnus albacares"],
    "region": "Arabian Sea"
  },
  "time_range": {
    "start": "2021-01-01",
    "end": "2025-12-31"
  },
  "variables": [
    "species_occurrence",
    "temperature"
  ],
  "requested_analysis": [
    "trend",
    "correlation"
  ]
}
```

Use the LLM only for interpreting language.

Validate the resulting structured query with Pydantic.

---

20. Dataset Routing

The router determines which data sources are required.

Example:

```
species trend
→ OBIS
→ FAO
→ ICES if available

temperature
→ Argo

taxonomy
→ WoRMS

molecular
→ MGnify
```

The routing layer should return:

```
{
  "required_sources": [
```

```
    "obis",
    "fishstat",
    "argo",
    "worms"
  ]
}
```

---

21. Agent Architecture

Create five logical agents.

21.1 Oceanography Agent

Responsibilities:

- temperature analysis
- salinity analysis
- oxygen analysis
- ocean anomalies
- depth patterns
- oceanographic evidence

Tools:

```
get_ocean_measurements
calculate_ocean_trend
calculate_ocean_anomaly
```

21.2 Fisheries Agent

Responsibilities:

- production
- catch
- fisheries trends
- species-related fisheries observations

Tools:

```
get_fisheries_records
calculate_fisheries_trend
compare_fisheries_periods
```

21.3 Biodiversity Agent

Responsibilities:

- species occurrence
- abundance
- spatial distribution
- species trends

Tools:

```
get_species_observations
get_species_distribution
calculate_biodiversity_trend
```

21.4 Molecular Biodiversity Agent

Responsibilities:

- molecular samples
- taxonomic profiles
- metagenomic evidence
- molecular biodiversity shifts

Tools:

```
get_molecular_samples
get_taxonomic_profile
compare_molecular_profiles
```

21.5 Synthesis Agent

Responsibilities:

- combine evidence
- identify supported hypotheses
- identify contradictions
- generate final answer
- preserve uncertainty
- include sources

The synthesis agent must NOT invent values not present in retrieved evidence.

---

22. Agent Communication Model

Each agent must return structured output, not unrestricted prose.

Example:

```
{
  "agent": "oceanography",
  "findings": [
    {
      "claim": "Regional mean temperature increased by 0.9°C.",
      "value": 0.9,
      "unit": "degC",
      "method": "linear trend",
      "confidence": 0.91
    }
  ],
  "evidence": [
    {
      "source": "Argo",
      "record_ids": ["..."],
      "description": "Regional monthly observations"
    }
  ]
}
```

This makes downstream reasoning much more reliable.

---

23. Claim-Level Reasoning

This is a core differentiating feature.

The system must decompose the proposed answer into atomic claims.

Example:

```
Claim 1:
Temperature increased.

Claim 2:
Species occurrence decreased.

Claim 3:
Fishing pressure increased.

Claim 4:
```

```
Temperature and species occurrence are negatively correlated.

Claim 5:
Temperature caused the decline.
```

Each claim must receive:

```
supported
partially_supported
contradicted
insufficient_evidence
```

The platform must never silently turn:

```
correlation
```

into:

```
causation
```

---

24. Evidence Graph

Model relationships:

```
Claim
  │
  ├── supported_by → Evidence A
  ├── supported_by → Evidence B
  ├── contradicted_by → Evidence C
  └── derived_from → Analysis D
```

Each evidence item should have:

```
source dataset
source record
analysis method
timestamp
location
value
```

---

25. Confidence Model

Confidence must be interpretable.

Initial MVP scoring may combine:

```
source reliability
data completeness
agreement among evidence
analysis robustness
claim specificity
```

Example conceptual score:

```
confidence =
0.30 * source_quality
+ 0.25 * evidence_support
+ 0.20 * cross_domain_support
+ 0.15 * data_completeness
+ 0.10 * analysis_quality
```

Treat these weights as configurable.

Do not present them as scientifically universal.

---

26. Final Answer Format

Every AI-generated response should follow:

```
INSIGHT

[2–5 sentence summary]

KEY FINDINGS

1. ...
2. ...
3. ...

EVIDENCE

- ...
- ...
- ...
```

```
RELATIONSHIPS

- ...
- ...

CONFIDENCE

High / Medium / Low

LIMITATIONS

- ...

SOURCES

- Argo
- OBIS
- FAO
- WoRMS
```

For a scientific question, include a clear statement such as:

> “The available data support an association but do not establish causation.”

where applicable.

---

27. Frontend

Use:

React + TypeScript

Recommended structure:

```
frontend/
  src/
    components/
    pages/
    hooks/
    services/
    types/
    maps/
    charts/
```

---

28. Main Pages

Dashboard

Display:

- number of datasets
- observations
- species
- active region
- latest anomaly count
- source status

Ask Marine Intelligence

Primary query interface:

```
Ask a question about marine ecosystems...
```

Suggested questions:

- “What changed in the Arabian Sea over the last five years?”
- “Which species are associated with this temperature anomaly?”
- “How has yellowfin tuna changed?”
- “What environmental factors correlate with this change?”

Oceanography Explorer

Display:

- temperature map
- salinity map
- depth
- time-series
- anomaly layer

Biodiversity Explorer

Display:

- species selector
- distribution map
- occurrence trends

- depth distribution

Fisheries Explorer

Display:

- production
- catch trends
- species
- region

Molecular Explorer

Display:

- sampling locations
- taxa
- abundance/profile
- time

Insight Page

Display:

- question
- generated insight
- confidence
- claims
- evidence
- source datasets
- maps
- charts

---

29. Map Requirements

Use:

MapLibre GL JS or another lightweight map library.

Required capabilities:

- pan
- zoom
- layer toggles

- markers
- heatmap
- choropleth
- time filtering
- tooltip

Layers:

```
Ocean temperature
Species occurrence
Fisheries
Molecular samples
Anomalies
```

---

30. Charts

Use a React-compatible charting library.

Required charts:

- line chart
- scatter plot
- bar chart
- heatmap
- area chart

The scatter plot should be able to display:

```
Temperature vs species occurrence
```

with:

- trend line
- correlation value
- sample count

---

31. Ecosystem Relationship Graph

Create an interactive graph:

```
Temperature
      ↓
Phytoplankton
      ↓
```

```
Fish abundance
      ↓
Fisheries pressure
```

Graph nodes represent:

- species
- environmental variables
- fisheries activities
- locations
- molecular entities

Edges represent:

- observed relationship
- correlation
- shared location
- shared time
- source-supported relationship

The UI must label whether an edge is:

```
observed
correlated
supported_by_source
```

Do not label a simple correlation as causal.

---

32. Backend

Use:

Python + FastAPI

Structure:

```
backend/
  app/
    main.py
    config.py

    api/
      routes/
        query.py
        datasets.py
        ocean.py
        biodiversity.py
```

```
        fisheries.py
        molecular.py
        analytics.py

    models/
    schemas/
    services/
      query_service.py
      analytics_service.py
      evidence_service.py
      taxonomy_service.py
      spatial_service.py

    agents/
      oceanography.py
      fisheries.py
      biodiversity.py
      molecular.py
      synthesis.py

    ingestion/
    repositories/
    utils/
```

---

33. API Endpoints

At minimum:

Query

```
POST /api/query
```

Request:

```
{
  "question": "Why has yellowfin tuna changed in the Arabian Sea?"
}
```

Response:

```
{
  "query_id": "...",
  "answer": "...",
  "confidence": 0.84,
  "claims": [],
```

```
  "evidence": [],
  "visualizations": []
}
```

Species

```
GET /api/species/{species_id}
GET /api/species/{species_id}/distribution
GET /api/species/{species_id}/trend
```

Ocean

```
GET /api/ocean/measurements
GET /api/ocean/trend
GET /api/ocean/anomalies
```

Fisheries

```
GET /api/fisheries/trend
GET /api/fisheries/species/{species_id}
```

Biodiversity

```
GET /api/biodiversity/observations
GET /api/biodiversity/distribution
```

Molecular

```
GET /api/molecular/samples
GET /api/molecular/profile
```

Evidence

```
GET /api/query/{query_id}/claims
GET /api/query/{query_id}/evidence
```

---

34. AI Provider Abstraction

Do not hard-code one LLM provider throughout the application.

Create:

```
class LLMProvider:
    def generate(...)
    def structured_output(...)
    def embed(...)
```

Implement a provider such as Gemini initially.

All secrets must come from environment variables.

Example:

```
LLM_API_KEY=
LLM_MODEL=
EMBEDDING_MODEL=
DATABASE_URL=
```

The entire application must continue functioning in non-LLM demo mode.

---

35. RAG

RAG should be used for:

- dataset descriptions
- scientific documentation
- metadata
- domain definitions
- explanatory source material

Do NOT use a vector database as the only place for numerical observations.

Numerical observations must remain queryable through PostgreSQL/scientific APIs.

Use:

```
SQL/PostGIS → numerical evidence
Vector search → textual/contextual evidence
```

Then combine both.

---

36. Provenance Requirements

Every displayed scientific value must be traceable to:

```
Dataset
Source record
Transformation
Analysis
Timestamp
```

Example:

```
Temperature trend: +0.9°C

Source:
Argo

Processing:
Regional monthly mean

Period:
2021–2025

Records:
12,481

Analysis:
Linear regression
```

---

37. Dataset Registry UI

Create an admin/internal page:

```
Dataset
Domain
```

```
Status
Records
Last updated
Geographic coverage
Temporal coverage
License
```

Example:

```
Argo
Oceanography
✓ Loaded
2.1M records
2025-...
Arabian Sea
2018–2025
```

---

38. Data Quality Checks

Every ingestion pipeline must calculate:

- total rows
- missing values
- duplicate records
- invalid coordinates
- invalid timestamps
- invalid units
- unknown species percentage

Example report:

```
OBIS ingestion

Rows processed: 2,481,002
Valid: 2,430,111
Invalid coordinates: 8,213
Unknown taxa: 23,110
Duplicates: 19,568
```

---

39. Security

Implement:

- `.env`
- no secrets committed
- input validation
- SQL parameterization
- request size limits
- rate limiting for public APIs if needed
- basic logging
- CORS configuration

Do not expose API keys to frontend JavaScript.

---

40. Performance

The MVP should:

- load dashboard data in under 3 seconds from local database for normal queries.
- return simple SQL analytics in under 2 seconds.
- return AI-assisted complex queries in a reasonable interactive window.
- cache frequent query results.
- avoid loading entire datasets into memory.
- use indexed database columns for:
- timestamp
- species_id
- location geometry
- dataset_id

---

41. Caching

Implement caching for:

- taxonomy lookups
- repeated analytics queries
- repeated AI query results
- external API responses

A simple Redis layer is optional.

Do not require Redis for the first runnable version.

Use an in-memory or database cache initially.

---

42. Logging

All major pipeline steps must log:

```
timestamp
request_id
operation
dataset
status
duration
error
```

AI requests must log:

```
query_id
agent
model
latency
token usage if available
```

Never log API secrets.

---

43. Error Handling

If Argo is unavailable:

```
Do not crash application.
```

Instead:

```
Argo unavailable
Using cached snapshot
```

If an entire data domain is unavailable:

```
Molecular data unavailable for this query.
The answer is based on oceanographic + biodiversity + fisheries evidence.
```

The AI must explicitly disclose missing evidence.

---

44. Demo Mode

Create an application-level:

```
DEMO_MODE=true
```

When enabled:

- load seeded local datasets
- disable reliance on external APIs
- use deterministic synthetic supplement
- use cached AI responses where useful

The final demo must work without internet access except where the team intentionally chooses a live API demonstration.

---

45. Three Official Demo Scenarios

Scenario A — Species decline

User:

> "How has yellowfin tuna changed in the Arabian Sea over the last five years?"

System:

1. identifies species
2. identifies region
3. identifies period
4. retrieves OBIS/Fisheries evidence
5. computes trend
6. displays distribution map
7. returns source-backed insight

Scenario B — Environmental relationship

User:

> "Could temperature changes be related to the observed change in yellowfin tuna?"

System:

1. retrieves species trend
2. retrieves Argo temperature trend
3. aligns time periods

4. computes correlation
5. presents scatter plot
6. provides correlation
7. explicitly states that correlation does not prove causation
8. gives evidence

Scenario C — Ecosystem reasoning

User:

> “What major ecosystem changes are occurring in this region?”

System:

1. retrieves ocean anomalies
2. retrieves biodiversity trends
3. retrieves fisheries changes
4. retrieves molecular evidence where available
5. runs four domain agents
6. extracts claims
7. finds supporting/contradicting evidence
8. synthesizes findings
9. displays ecosystem graph
10. provides a confidence score

---

46. Evaluation Framework

The system must be evaluated quantitatively.

Data quality

Measure:

```
ingestion success rate
normalization success rate
invalid-record rate
duplicate rate
```

Taxonomy

Measure:

```
correct species mapping %
unresolved species %
```

Retrieval

Measure:

```
Precision@K
Recall@K
```

Scientific analytics

Compare outputs against independent manual/calculated baselines.

Claim verification

Measure:

```
claim correctness
claim grounding
citation correctness
unsupported claim rate
```

Important metric:

```
Unsupported Claim Rate =
unsupported claims / total generated claims
```

Aim for the lowest possible unsupported-claim rate.

---

47. Answer Quality Benchmark

Create at least 30 manually authored test queries.

Group them into:

```
10 oceanographic
10 fisheries/biodiversity
5 molecular
5 cross-domain
```

For each query, define:

```text expected datasets expected variables expected analysis expected source expected reasoning limitation

```
Do not compare only final prose.

Compare:

- retrieved evidence
- computed numbers
- claims
- source attribution

---

48. Test Dataset

Create a version-controlled evaluation dataset:

```text
evaluation/
  questions.json
  expected_sources.json
  expected_claims.json
```

Example:

```
{
  "question": "How has yellowfin tuna changed...",
  "required_sources": ["OBIS", "FAO", "Argo"],
  "required_analysis": ["trend", "correlation"],
  "forbidden_claims": ["causal certainty"]
}
```

---

49. Automated Tests

Must include:

Unit tests

For:

- taxonomy normalization
- unit conversion
- spatial mapping

- temporal aggregation
- analytics
- scoring

Integration tests

For:

```
ingestion → DB
DB → analytics
analytics → agent
agent → synthesis
API → frontend
```

AI evaluation tests

For:

- unsupported claim detection
- source consistency
- structured output validity

---

50. Team Responsibilities

The six-person development split should be:

Person 1 — Data Engineering

Own:

- Argo ingestion
- OBIS ingestion
- FAO ingestion
- MGnify ingestion
- dataset registry
- raw/processed data management

Person 2 — Database + Standardization

Own:

- PostgreSQL/PostGIS
- schema
- WoRMS integration
- taxonomy normalization

- spatial normalization
- temporal normalization
- indexes

Person 3 — Scientific Analytics

Own:

- trends
- anomaly detection
- correlations
- spatial analysis
- statistical validation
- analytics APIs

Person 4 — AI / Agents

Own:

- query understanding
- routing
- domain agents
- evidence fusion
- claim extraction
- claim verification
- synthesis

Person 5 — Frontend / Visualization

Own:

- React application
- maps
- charts
- ecosystem graph
- query interface
- evidence interface

Person 6 — Integration / QA / Demo

Own:

- service integration
- test suite

- performance
- demo scenarios
- evaluation
- deployment
- documentation
- presentation support

Person 6 must also maintain the canonical API contract shared by everyone.

---

51. Repository Structure

Use a monorepo:

```
marine-intelligence/
│
├── backend/
│   ├── app/
│   ├── ingestion/
│   ├── agents/
│   ├── analytics/
│   ├── database/
│   ├── tests/
│   └── requirements.txt
│
├── frontend/
│   ├── src/
│   ├── public/
│   └── package.json
│
├── data/
│   ├── raw/
│   ├── processed/
│   ├── demo/
│   └── metadata/
│
├── evaluation/
│   ├── questions.json
│   ├── expected_claims.json
│   └── benchmarks/
│
├── scripts/
│   ├── ingest_all.py
│   ├── generate_demo_data.py
│   └── seed_database.py
│
├── docs/
│   ├── architecture.md
```

```
│   ├── dataset_registry.md
│   └── api.md
│
├── docker-compose.yml
├── .env.example
├── README.md
└── Makefile
```

---

52. Local Development

The complete MVP should launch using:

```
docker compose up
```

Services:

```
frontend
backend
postgres
```

Redis should be optional.

The application must provide:

```
make setup
make seed
make dev
make test
```

---

53. Environment Variables

Create `.env.example`:

```
DATABASE_URL=
LLM_API_KEY=
LLM_MODEL=
EMBEDDING_MODEL=

ARGO_ENABLED=true
OBIS_ENABLED=true
FISHSTAT_ENABLED=true
```

```
MGNIFY_ENABLED=true
WORMS_ENABLED=true

DEMO_MODE=false

DEFAULT_REGION=
DEFAULT_START_DATE=
DEFAULT_END_DATE=
```

Never commit `.env`.

---

54. Implementation Order

Claude Code must implement in this order.

Phase 1 — Foundation

Build:

- repository
- Docker
- PostgreSQL/PostGIS
- FastAPI
- React
- health-check endpoint
- database migrations
- basic CI/test setup

Acceptance:

```
docker compose up
```

works from a clean checkout.

Phase 2 — Unified schema

Implement:

- datasets
- species
- locations
- ocean measurements
- biodiversity
- fisheries
- molecular samples

- evidence
- claims

Seed synthetic data.

Acceptance:

A SQL query can retrieve cross-domain observations for the same region/species/time period.

Phase 3 — Data ingestion

Implement:

1. WoRMS
2. Argo
3. OBIS
4. FAO
5. MGnify

In this order.

Every connector must work independently.

Phase 4 — Analytics

Implement:

- trend
- correlation
- anomaly
- spatial distribution
- regional aggregation

Expose REST APIs.

Phase 5 — AI Query Layer

Implement:

- query interpretation
- Pydantic structured query
- dataset routing
- evidence retrieval

Do NOT add complex multi-agent behavior yet.

First make:

```
question → data → analysis → answer
```

work.

Phase 6 — Agents

Add:

- oceanography agent
- fisheries agent
- biodiversity agent
- molecular agent
- synthesis agent

Each returns structured JSON.

Phase 7 — Claim/Evidence Layer

Implement:

```
response
 ↓
claims
 ↓
evidence
 ↓
support/contradiction
 ↓
confidence
```

Integrate into final synthesis.

Phase 8 — Frontend

Build:

- dashboard
- map
- charts
- query UI
- insight page
- evidence panel
- ecosystem graph

Phase 9 — Evaluation

Create:

- benchmark questions
- metrics
- evaluation dashboard
- error cases

Phase 10 — Demo Hardening

Implement:

- demo mode
- local data snapshots
- graceful API failure
- loading states
- error handling
- cached responses
- polished UI

---

55. MVP Definition

The MVP is considered complete when all of these work:

Data

- Argo subset loaded.
- OBIS subset loaded.
- FAO fisheries subset loaded.
- WoRMS taxonomy integrated.
- MGnify represented or integrated for at least one molecular workflow.

Analytics

- species trend works.
- temperature trend works.
- correlation works.
- spatial map works.

AI

Natural-language question can be translated into a structured query.

At least three domain agents can return structured evidence.

Synthesis agent generates a grounded answer.

Claims have evidence.

UI

User can:

- ask question
- see answer
- see confidence
- see sources
- see map
- see chart
- inspect evidence

Reliability

System works in demo mode without external internet dependencies.

---

56. Advanced Features After MVP

Only build after MVP works.

Potential extensions:

- Copernicus Marine data
- ICES DATRAS
- EMODnet Biology
- GBIF integration
- Neo4j knowledge graph
- more sophisticated graph reasoning
- agent reputation
- explicit agent disagreement resolution
- automated hypothesis generation
- temporal forecasting
- ecosystem digital twin
- real-time alerts
- user accounts
- saved research sessions

These are NOT MVP requirements.

---

57. Important Scientific Guardrails

The application must distinguish:

Observation

> “Temperature increased by 1.1°C.”

Statistical relationship

> “Temperature and species occurrence show a negative correlation.”

Hypothesis

> “The temperature increase may be associated with the observed species decline.”

Causal claim

> “Temperature caused the decline.”

Only the first three are appropriate unless the system has a causal methodology capable of supporting the fourth.

The system should default to cautious language.

---

58. Important AI Guardrails

The LLM must never:

- fabricate dataset values.
- fabricate citations.
- fabricate species observations.
- invent correlations.
- invent source URLs.
- claim to have queried a source when it did not.
- turn missing data into assumed data.

When evidence is insufficient, output:

> **Insufficient evidence to answer confidently.**

---

59. Product Differentiator

The application should be positioned around three ideas:

Unified

Multiple marine data domains become interoperable through:

- taxonomy

- geography
- time
- common observation structures

Intelligent

AI agents reason over domain-specific evidence.

Trustworthy

Every substantive claim is associated with:

- source
- evidence
- calculation
- confidence
- limitation

The final positioning:

> **A trustworthy marine intelligence layer for cross-domain ecosystem reasoning.**

---

60. Final User Experience

A user should be able to open the application and enter:

> **“What is happening to marine biodiversity in the Arabian Sea?”**

The system should produce something like:

```
MARINE ECOSYSTEM INSIGHT

Three significant changes were detected in the selected
region during the analysis period.

1. Ocean temperature anomaly
   Temperature increased relative to the selected baseline.

2. Biodiversity change
   Several species show changes in occurrence/distribution.

3. Fisheries change
   Fisheries indicators also changed during the period.

POSSIBLE RELATIONSHIPS

Temperature and biodiversity changes show a statistical
```

```
association in the selected period.

Fisheries pressure is another potentially relevant factor.

The available data do not establish that temperature
alone caused the biodiversity change.

CONFIDENCE
Moderate

DATA SOURCES
✓ Argo
✓ OBIS
✓ FAO
✓ WoRMS
✓ MGnify

[View Map]
[View Evidence]
[View Trends]
[View Claims]
```

---

61. Final Engineering Instruction to Claude Code

Build this system **incrementally and keep every stage runnable**.

Do not implement the full architecture as a single giant change.

At the end of every phase:

1. Run the tests.
2. Run the application.
3. Verify the API.
4. Verify the database.
5. Verify the frontend.
6. Update documentation.
7. Preserve backward compatibility.

When a public dataset is unavailable, **do not block implementation**. Build the connector interface and provide a deterministic local snapshot or synthetic fixture.

When an external API changes, isolate the change inside its connector rather than spreading source-specific logic throughout the application.

Do not introduce additional frameworks merely because they are popular.

Prefer:

```
PostgreSQL + PostGIS
FastAPI
Python
React + TypeScript
Pydantic
Pandas / xarray
LLM API
Docker
```

Keep the system modular enough that future datasets can be added by implementing one new connector.

The highest-priority requirement is:

> **The system must demonstrate genuine cross-domain reasoning over real, traceable marine data rather than merely wrapping an LLM around a dashboard.**

The second-highest priority is:

> **Every important AI-generated claim must be traceable to data, analysis, or an explicit source.**

The third-highest priority is:

> **The entire demo must run reliably using local snapshots/demo data even if external datasets or APIs are unavailable.**